

Apache Flink: The Model and Its Edges

What it computes, why Kubernetes suits it, and where the abstraction stops holding

flink-stream reference environment

11 September 2026

Contents

1	The problem Flink exists to solve	2
1.1	What came before, and why it was unsatisfying	3
2	How Flink works	3
2.1	The dataflow graph	3
2.2	The two processes	3
2.3	State, and why it is keyed	4
2.4	Time and watermarks	4
2.5	Checkpoints	5
2.6	Exactly-once past the edge of the system	5
2.7	Backpressure	6
3	The abstractions Flink embraces	6
3.1	Layered APIs over one runtime	6
3.2	Stream-table duality	6
3.3	Batch as a special case	6
3.4	Event time as the primary clock	7
3.5	State as a local, first-class, checkpointed thing	7
4	Where Kubernetes fits	7
4.1	What a Flink cluster needs from its environment	7
4.2	Native integration	7
4.3	Why an operator is necessary, not merely convenient	7
4.4	What the operator adds beyond lifecycle	8
4.5	Where Kubernetes and Flink disagree	8
5	What Flink does not address	9
5.1	It is not a serving layer	9
5.2	It is not an interactive analytics engine	9
5.3	It does not reach below a latency floor	9
5.4	It is an expensive way to do something simple	10
5.5	It does not host thousands of small jobs well	10
5.6	The job graph is fixed at submission	10
5.7	Machine learning is not its domain	10

5.8	Very large reference data is someone else's problem	10
5.9	Global, multi-region state is not modelled	10
6	Leaky abstractions	11
6.1	"It's just SQL" – but the query plan is the API	11
6.2	"Event time makes results deterministic" – watermarks are a guess	11
6.3	"Exactly-once" – three separate promises wearing one name	12
6.4	"State is managed for you" – until the schema changes	12
6.5	"Keying distributes the work" – until the keys are skewed	12
6.6	"Batch and streaming are unified" – one API, two behaviours	13
6.7	"The runtime handles failures" – recovery is global	13
6.8	"Backpressure is handled" – it converts into a different failure	13
6.9	"The operator manages the job" – it does not understand the job	13
6.10	"Connectors are interchangeable" – each has its own semantics	14
7	A short summary	14
8	Appendix: where these concepts appear in the reference environment	14

1 The problem Flink exists to solve

Most data infrastructure assumes the data has stopped arriving. A query runs against a table that is not changing; a job reads a directory that is complete; a report covers a day that has ended. The assumption is so deeply built in that it is rarely stated.

Streaming systems exist because for a growing class of work that assumption is false and waiting for it to become true is the whole cost. Fraud detection that fires tomorrow morning is not fraud detection. A per-minute revenue figure computed hourly is an hourly figure. The interesting question is not "how do we process data faster" but "**how do we produce a correct answer over data that has not finished arriving, and keep the answer correct as the rest of it shows up?**"

That question has three hard parts, and Flink is essentially an answer to all three at once.

Time is not arrival order. Events happen in one order and arrive in another – a mobile client buffers offline, a shard lags, a retry resends. A system that groups by arrival time computes something, but not what anyone asked for. Correctness requires reasoning in the time the events *happened*, which means deciding how long to wait for stragglers and what to do about the ones that arrive anyway.

Computation over a stream is stateful. A running count, a session, a join, a pattern – each needs memory of what came before. That memory is often larger than a single machine, must survive process death, and must be redistributable when the job is resized. State is not an implementation detail of stream processing; it is the substance of it.

Failure is continuous, not exceptional. A job that runs for a year will lose machines. Recovery cannot mean "start over", because there is no beginning to return to, and it cannot mean "lose a bit", because the output is usually feeding something that counts money.

1.1 What came before, and why it was unsatisfying

The Lambda architecture – a fast approximate streaming path beside a slow correct batch path – was the industry’s honest admission that no single system did both. It worked, at the cost of writing and reconciling every piece of business logic twice. The reconciliation was the part that never stopped costing.

Micro-batching (Spark Streaming’s original model) collapses the two by making streaming a fast sequence of small batches. That buys the batch engine’s correctness properties, but time is still defined by batch boundaries: latency has a floor at the batch interval, event time is awkward, and a session that spans batches has to be stitched back together.

Flink’s bet was the other direction: make the streaming model primitive and treat a bounded dataset as a stream that happens to end. Everything distinctive about the system follows from taking that seriously.

2 How Flink works

2.1 The dataflow graph

A Flink program is a directed graph of operators. Sources produce records, transformations consume and emit them, sinks terminate. The program you write is a *logical* graph; the runtime compiles it to a physical one in which each operator becomes some number of parallel **subtasks**, each handling a slice of the data.

Between operators the runtime chooses a distribution pattern:

Pattern	Meaning	Cost
Forward	subtask n feeds subtask n	none; chainable
Hash (keyBy)	destination chosen by key	network shuffle, serialization
Rebalance / rescale	round-robin	network, but no key semantics
Broadcast	every record to every subtask	n -fold duplication

Adjacent operators with a forward connection and matching parallelism are **chained** into one task: no serialization, no thread handoff, one function call. This is why an unnecessary keyBy is expensive in a way that is invisible in the source code – it does not add work, it breaks a chain and inserts a network boundary.

A **task slot** on a TaskManager holds one such pipeline slice. Slot sharing lets one slot hold one subtask of each operator in the job, so a job with parallelism p generally needs p slots regardless of how many operators it has.

2.2 The two processes

Process	Owns
JobManager	the job graph, scheduling, checkpoint coordination, failure recovery, the REST API and web UI
TaskManager	slots, the actual operator execution, local state, network buffers, shuffles

In **application mode** – the mode that matters for anything long-running – the job’s `main()` runs on the JobManager and the cluster exists for exactly one job. In session mode a standing cluster accepts submitted jobs, which is convenient for interactive SQL and wrong for production pipelines, because one job’s memory exhaustion takes down every other job sharing the TaskManager.

2.3 State, and why it is keyed

Flink state comes in two shapes. **Operator state** is per-subtask – a Kafka source’s partition offsets are the canonical example. **Keyed state** is the important one: it is scoped to the key of the record currently being processed, so a function that calls `valueState.value()` gets the value for *this* key without ever naming it.

That scoping is what makes state redistributable. Keys are hashed into a fixed number of **key groups**, and key groups – not individual keys – are dealt out to subtasks in contiguous ranges. The count is `pipeline.max-parallelism`, and the entire rescaling story is this: changing parallelism re-deals existing key-group ranges to a different number of subtasks. Nothing is recomputed.

```
keyGroup = murmurHash(key.hashCode()) % maxParallelism
subtask  = keyGroup * parallelism / maxParallelism
```

Two state backends matter. `HashMapStateBackend` keeps objects on the JVM heap: fastest per access, bounded by memory, full snapshots. `EmbeddedRocksDBStateBackend` keeps serialized bytes in an embedded LSM store on local disk, supports state far larger than memory, and – critically – supports **incremental checkpoints**, uploading only the SST files that changed. A hundred-megabyte state does not cost a hundred megabytes of upload every checkpoint interval.

Checkpoint *storage* is a separate decision from the backend. The snapshot has to land somewhere durable and shared – object storage in any real deployment. A local volume is a checkpoint that dies with its node.

2.4 Time and watermarks

Flink distinguishes **event time** (when the thing happened, carried in the record) from **processing time** (when the operator saw it). Only event time gives reproducible results: a replay of yesterday’s topic produces yesterday’s answers.

The mechanism is the **watermark** – an assertion, injected into the stream alongside records, that no event with a timestamp earlier than *T* is still expected. Watermarks flow through the graph and drive everything with a temporal boundary: window firing, join buffer eviction, timer callbacks, pattern timeouts, state expiry.

Two properties of watermark propagation explain most of the surprises:

- An operator with several inputs advances to the **minimum** of its inputs' watermarks. One slow input holds back the entire downstream graph. This is correct – you cannot close a window while an input might still deliver into it – and it is also why a single idle Kafka shard freezes event time for a whole job.
- The watermark is a *heuristic* supplied by you, not a fact derived from the data. `forBoundedOutOfOrderness(5s)` is a guess about how unordered the stream is. Guess low and correct records are dropped as late; guess high and every result is delayed by the margin.

2.5 Checkpoints

Flink's fault tolerance is asynchronous barrier snapshotting – a variant of Chandy-Lamport adapted to a dataflow graph.

The coordinator injects a numbered **barrier** into every source. Barriers flow with the records. When an operator has received barrier *n* on all of its inputs, it snapshots its state and forwards the barrier downstream. When every operator has acknowledged, checkpoint *n* is complete and represents a globally consistent cut of the entire pipeline: all state as of exactly the same set of input records.

Recovery is then simple to state: reset every operator to the last complete checkpoint, rewind every source to the offsets recorded in it, and resume. Records between the checkpoint and the failure are replayed. This gives **exactly-once state** – each record affects state once – and it requires only that sources be replayable.

Waiting for a barrier on every input is **alignment**, and it is where a wide shuffle stalls: the fastest input blocks while the slowest catches up. **Unaligned checkpoints** let barriers overtake queued records at the cost of persisting those in-flight records into the snapshot. The usual configuration is to stay aligned until alignment is measurably slow:

```
execution.checkpointing.unaligned.enabled: "true"
execution.checkpointing.aligned-checkpoint-timeout: "10s"
```

A **savepoint** is the user-owned cousin of a checkpoint: canonical format, self-contained, and restorable into a *different* parallelism or a *modified* job graph. Checkpoints are the runtime's recovery mechanism; savepoints are the deployment mechanism.

2.6 Exactly-once past the edge of the system

Exactly-once state is internal. Getting it end-to-end requires the sink to participate, and the protocol is two-phase commit: the sink opens a transaction per checkpoint interval, pre-commits on snapshot, and commits when the checkpoint completes.

The consequence is structural and often unwelcome: **output becomes visible on checkpoint completion**, so a thirty-second checkpoint interval means up to thirty seconds of output latency for a `read_committed` consumer. That is not tuning overhead. It is the price of the guarantee, and it is the correct moment to ask whether at-least-once with an idempotent sink would serve better.

Sinks that cannot do transactions do not get the guarantee. Flink's JDBC connector can use XA two-phase commit where the driver exposes an `XADataSource`; where it does not – YugabyteDB, in this project – the honest answer is at-least-once delivery plus `INSERT ... ON CONFLICT DO UPDATE` on a natural key, making the *result* idempotent rather than the *delivery* unique.

2.7 Backpressure

Flink’s network stack uses credit-based flow control: a receiver advertises buffer credits, a sender only transmits what it has credit for. When an operator slows down, its input buffers fill, credits stop being issued, and the slowdown propagates upstream to the source, which simply reads Kafka more slowly.

This is a genuinely good design – there is no queue to overflow and no data to drop – and it has one important interaction: under sustained backpressure, barriers travel slowly too, so checkpoints take longer and eventually time out. A job that is merely slow degrades into a job that cannot checkpoint, and a job that cannot checkpoint cannot commit transactional output. Backpressure is therefore not only a throughput symptom; it is a correctness-adjacent one.

3 The abstractions Flink embraces

3.1 Layered APIs over one runtime

Layer	You control	You give up
ProcessFunction	state, timers, side outputs, per-record logic	everything the optimizer would have done
DataStream	typed transformations, explicit keying and windows	plan-level rewriting
Table / SQL	declarative queries; planner picks the physical plan	operator identity and direct state control

All three compile to the same operator graph and interoperate within a job. This is the most useful structural property of the system: the fallback from “SQL cannot express this” is not a different framework, it is a different function in the same job, sharing the same checkpoint.

3.2 Stream-table duality

The Table API’s foundation is that a stream and a table are two views of the same thing. A table is the accumulated result of a changelog stream; a stream is the sequence of changes to a table. Flink makes this literal: every Table API query is compiled into operators exchanging **changelog** records tagged as insert, update-before, update-after, or delete.

This is why `CREATE TABLE` over a Kafka topic is not a metaphor, and also why the error does not support consuming update changes is so common – it is the type system telling you that an updating query has been pointed at an append-only sink.

3.3 Batch as a special case

Flink treats a bounded source as a stream with a known end. The same job, the same operators, the same SQL; the scheduler notices boundedness and switches to blocking shuffles, and the watermark advances to infinity at the end of input so every window fires. One dialect, one runtime, for both backfill and live.

The honesty required here: *unified* means one API and one runtime, not identical performance or identical semantics. The differences are real and are discussed under leaky abstractions below.

3.4 Event time as the primary clock

Making event time the default is the choice that most distinguishes Flink from systems that bolt it on. It means results are a function of the data and not of when the job happened to run – so a replay is reproducible, a backfill agrees with the live pipeline, and a slow consumer does not change the answer.

3.5 State as a local, first-class, checkpointed thing

The decision not to keep state in an external database is what makes the throughput numbers possible. State lives on the same machine as the computation, is accessed without a network hop, and is made durable asynchronously in the background. The entire checkpointing apparatus exists to buy back the durability that locality gives up.

4 Where Kubernetes fits

4.1 What a Flink cluster needs from its environment

Strip away the specifics and a Flink deployment needs: long-running supervised processes, resource limits and isolation, service discovery between JobManager and TaskManagers, distributed configuration and secrets, leader election and durable metadata for high availability, local scratch disks for RocksDB, and an API for adding and removing workers.

That list is very nearly a description of what Kubernetes is. The fit is not a coincidence of fashion; Flink's resource-management layer was already an abstraction over YARN and Mesos, and Kubernetes is a better-shaped target than either.

4.2 Native integration

Flink can talk to the Kubernetes API itself. In `kubernetes-application` mode the JobManager requests TaskManager pods directly from the API server as the job's resource requirements demand, rather than sitting inside a fixed standalone cluster someone provisioned. High availability uses Kubernetes primitives too – leader election through leases, JobManager metadata in ConfigMaps, with the actual state in the checkpoint directory.

4.3 Why an operator is necessary, not merely convenient

This is the part worth being precise about, because it explains why Flink on Kubernetes is not just a Deployment with a container image.

A stateful streaming job cannot be updated by a rolling restart. Kubernetes' native update strategies assume replicas are fungible and independently replaceable. Flink TaskManagers are neither: they are shards of a single distributed computation holding disjoint slices of keyed state, participating in a coordinated snapshot protocol. Replacing one at a time does not produce a gradually-updated job; it produces a job that restarts repeatedly from checkpoints, each time with a different half of the pipeline running the wrong code.

The correct update sequence is: stop the job with a savepoint, replace the image or configuration, restart from the savepoint. That is a Flink-specific procedure, it involves waiting for an asyn-

chronous operation to succeed, and it has to handle the cases where the savepoint fails or the new version does not start. Encoding that sequence is exactly what an operator is for.

```
spec:
  job:
    upgradeMode: savepoint    # stop-with-savepoint, restart from it
    # last-state              # from last checkpoint; safe only if graph unchanged
    # stateless               # discard state; the logic changed incompatibly
```

The operator’s reconciliation loop then distinguishes two independent status axes – what the operator is doing (`status.lifecycleState`) and what Flink is doing (`status.jobStatus.state`) – because “the operator believes it has deployed the job” and “the job is running” are genuinely different facts.

4.4 What the operator adds beyond lifecycle

Autoscaling that understands dataflow. A Horizontal Pod Autoscaler scales replicas on CPU. That is the wrong signal and the wrong granularity for a streaming job: the meaningful signals are source lag and per-vertex busyness, and the meaningful unit is the *operator*, not the job. The Flink autoscaler reads those metrics and rewrites parallelism per vertex, then performs a savepoint-and-restore to apply it. Advisory mode makes it a recommendation engine first:

```
job.autoscaler.enabled: "true"
job.autoscaler.scaling.enabled: "false"
```

Savepoints as Kubernetes objects. Periodic and on-demand savepoints become CRs with retention policy, which puts the disaster-recovery artifact in the same declarative system as everything else.

Restart semantics for deployment-level failures. Flink’s own restart strategy covers job failures. It does not cover a failure in `main()` – which, in application mode, runs on the JobManager and connects to Kafka, Schema Registry and sinks immediately. A dependency that is not yet up throws before the job exists, so no Flink restart strategy applies. The answer lives in the deployment: an init container that waits, plus

```
kubernetes.operator.job.restart.failed: "true"
```

4.5 Where Kubernetes and Flink disagree

Being specific about the friction is more useful than celebrating the fit.

Kubernetes assumes	Flink actually is
Pods are disposable and interchangeable	A TaskManager holds an irreplaceable slice of keyed state
Losing a replica degrades capacity	Losing a TaskManager restarts the job from a checkpoint
Liveness probes should restart unhealthy pods	A JobManager mid-recovery looks unhealthy and must not be restarted
HPA scales on CPU	Parallelism changes require a savepoint-restore cycle

Kubernetes assumes	Flink actually is
Local storage is ephemeral and unimportant	RocksDB performance is a direct function of local disk
Memory limits apply to the process	The JVM must be told the limit, or the kernel enforces it by killing

The memory row deserves its own note because it is the most common production incident. Flink’s memory model divides a process budget into framework heap, task heap, managed memory (off-heap, used by RocksDB), network buffers, metaspace and overhead. The cgroup limit counts *all* of it. Configuring heap against the pod’s memory limit – forgetting that RocksDB allocates natively outside it – produces an `OOMKilled` pod with no Java stack trace and nothing in the logs, because the kernel does not ask the JVM first.

Spot instances and aggressive node autoscaling deserve the same scrutiny. Both are excellent for stateless workloads and both convert, for a stateful Flink job, into “restart the entire pipeline from the last checkpoint, at an interval chosen by the cloud provider.”

5 What Flink does not address

A model is easier to use well when its boundary is explicit. None of the following are defects; they are places where the shape of the tool does not match the shape of the problem.

5.1 It is not a serving layer

Flink computes results. It does not serve them. There is no supported way to ask a running job “what is the current value for key k ” – Queryable State was an experiment that never reached production quality and has been retired. Results reach consumers by being written to something built for reads.

This is the single most common architectural mistake made by teams new to the system: treating job state as a database because it is, technically, a store of keyed values. It has no secondary indexes, no ad-hoc query language, no access path except the key the operator is currently processing, and no read availability during recovery. The reference environment’s pattern – compute in Flink, write to YugabyteDB and Iceberg, serve from there – is the normal one and not a workaround.

5.2 It is not an interactive analytics engine

Flink SQL over historical data works, and it will lose badly to a system designed for it. A columnar warehouse or an OLAP store (Trino, ClickHouse, Druid, Pinot, Snowflake) has indexes, statistics, vectorized execution and a cost model tuned for scanning old data; Flink’s optimizer is tuned for incremental maintenance of a result. Ad-hoc exploration also implies constantly changing queries, which – as the next section shows – is precisely the workload Flink’s state model handles worst.

5.3 It does not reach below a latency floor

Flink’s per-record latency is milliseconds, which is fast but is not “real time” in the sense a control-systems or trading engineer means. The floor is set by network buffer timeouts, JVM garbage collection pauses, and – if end-to-end exactly-once is required – the checkpoint interval, since output

only becomes visible on commit. Requirements in the microsecond range, or hard latency *bounds* rather than low averages, want a different class of system.

5.4 It is an expensive way to do something simple

A Flink deployment is a JobManager, TaskManagers, checkpoint storage, an operator, metrics, and people who understand watermarks. For a stateless map-and-filter from one topic to another at modest volume, a Kafka consumer loop or Kafka Streams delivers the same result with a fraction of the operational surface. The threshold where Flink starts paying for itself is roughly: state that does not fit in one process, or event-time correctness that actually matters, or enough pipelines to amortize the platform.

5.5 It does not host thousands of small jobs well

The unit of isolation is a cluster per job. A hundred tenants with a pipeline each means a hundred JobManagers, a hundred checkpoint streams and a hundred sets of TaskManager overhead. Session mode shares the cluster but shares the failure domain with it. The idiomatic answer is one job whose *logic* is parameterized – a broadcast stream of rules, or dynamic CEP patterns – which works, and which also means that changing one tenant’s rule is a change to a shared job.

5.6 The job graph is fixed at submission

A running job’s topology cannot change. Adding an operator, changing a window type, or repartitioning means a new job graph and a redeploy. Systems whose requirements include operators appearing and disappearing at runtime are fighting the model; what Flink offers instead is data-driven behaviour inside a fixed graph.

5.7 Machine learning is not its domain

Flink ML and the older iteration APIs exist, and Flink is genuinely good at *feature computation* and at *applying* a model per record. Training is another matter: iterative algorithms over a fixed dataset are the workload the DataSet API was built for and subsequently deprecated, and no one should choose Flink over a purpose-built training stack for it.

5.8 Very large reference data is someone else’s problem

Enriching a stream against a hundred-gigabyte dimension table has two options. Broadcast it – every subtask holds a full copy, which multiplies the memory by the parallelism. Or use a lookup join against an external store, which reintroduces a network dependency, a failure mode and a latency the local-state model was designed to eliminate. Neither is wrong; both are compromises the abstraction does not hide.

5.9 Global, multi-region state is not modelled

Flink has no notion of a job spanning regions with partitioned state and a consistency story between them. A multi-region deployment is multiple independent jobs plus a replication design that is entirely yours.

6 Leaky abstractions

These are the places where the model is clean, the implementation underneath is visible anyway, and the gap is where production incidents come from. Every one of them has been hit in this reference environment.

6.1 “It’s just SQL” – but the query plan is the API

In a database, a query is a request and the plan is an implementation detail free to change. In Flink SQL, the plan is a long-lived stateful process, and the *structure of the plan* determines the operator identities against which state is stored.

The consequence is severe: **editing a query renames operators, and restoring a savepoint into the edited query either fails or silently starts from empty state**. A join that was accumulating six hours of history quietly begins with none, no error is raised, and the output is wrong in a way no test catches. Flink offers `COMPILED PLAN` to pin the physical plan across upgrades; until that is in use, a query edit must be treated as a stateless redeploy.

The corresponding `DataStream` leak is the same mechanism seen from the other side. Operator `UIDs` are derived from graph position unless set explicitly, so inserting an operator anywhere upstream renames everything downstream, with the same silent empty-state restore. Every stateful operator sets one by hand, forever:

```
.process(new AccountJoinFunction(config.stateTtl()))
    .name("H: join ref.accounts (versioned, TTL'd)")
    .uid("lab-h-account-join");
```

6.2 “Event time makes results deterministic” – watermarks are a guess

Event time is presented as a clean semantic: results depend on when events happened. The leak is that the clock is supplied by a heuristic you configure, and the heuristic fails in ordinary conditions.

Idle sources. A source subtask reading several shards emits the minimum watermark across them. One quiet shard pins event time for the entire job and no window ever fires. On a lightly loaded cluster this is the normal state, not an edge case – and the symptom is a perfectly healthy job producing nothing at all.

```
WatermarkStrategy.forBoundedOutOfOrderness(Duration.ofSeconds(5))
    .withIdleness(Duration.ofSeconds(15));
```

```
SET 'table.exec.source.idle-timeout' = '15 s';
```

Lateness is silent by default. Records arriving after their window’s watermark are dropped, and the drop is a metric, not an error. A side output for late data turns a silent correctness loss into an observable one, and is worth the few lines it costs.

Replay does not reproduce watermarks. Reprocessing from the earliest offset reads historical data as fast as the network allows, so the out-of-orderness bound that was generous in real time is meaningless when a day passes in ten seconds. Backfills need different watermark settings, or watermark alignment across sources, or batch mode.

6.3 “Exactly-once” – three separate promises wearing one name

Exactly-once *state* is delivered by checkpointing and is essentially free. Exactly-once *end-to-end* is a property of the sink, not of Flink, and it leaks in three directions.

Latency. Output appears at checkpoint commit. With a thirty-second interval, a `read_committed` consumer sees output up to thirty seconds late. A healthy job with an apparently empty output topic is almost always this.

Coupling to broker configuration. `transaction.timeout.ms` must comfortably exceed the checkpoint interval, or the broker expires the transaction and the job fails on commit. A failed checkpoint is not just a lost checkpoint; it is output that never becomes visible.

Identifier collisions. Transactional ids are derived as `prefix-subtask-checkpoint`. Two sinks writing the same table with the same prefix generate identical ids, and Kafka fences one:

```
ProducerFencedException: There is a newer producer with the same
                           transactionalId which fences the current one
```

The job then restart-loops. Two INSERT statements into one exactly-once Kafka table is always wrong.

Not every sink can offer it. The guarantee depends on the destination system’s transactional capabilities, which means the architecture diagram’s “exactly-once” label is really a claim about the last hop, not about Flink.

6.4 “State is managed for you” – until the schema changes

Managed state hides serialization, redistribution and durability, and then exposes all three at once at upgrade time.

Serializer evolution. State is stored as bytes written by a specific serializer. Adding a field to a POJO or an Avro record held in state requires the serializer to support schema evolution; if it does not, restore fails, and the diagnosis arrives during a deployment rather than during development. Kryo’s fallback serializer, chosen automatically for types Flink cannot analyze, does not support evolution at all – so a type that quietly falls back to Kryo is a future failed upgrade.

maxParallelism is frozen on first checkpoint. It is the key-group count, it is baked into the state snapshot, and changing it invalidates that state. The default is derived from the initial parallelism, which means an unconsidered first deployment can cap a job’s future scaling permanently. It is chosen once, with headroom, before the first production run.

Unbounded state is the default. A regular JOIN in Flink SQL retains both sides forever, because correctness requires it – a row arriving in a year might match. The abstraction says “join”; the implementation says “keep everything”. Without `table.exec.state.ttl` the disk fills, and TTL is not a tuning parameter but a deliberate choice to be wrong about very old matches in exchange for finite storage.

6.5 “Keying distributes the work” – until the keys are skewed

The keyed-state model promises a partitioned computation. It does not promise the partitions are equal. One hot key – a dominant merchant, a test account, a null placeholder – lands entirely in one subtask, and that subtask becomes the job’s throughput. No amount of parallelism helps, because

the unit of assignment is the key. The fixes are all application-level: salt the key and aggregate twice, or two-phase the aggregation so that a local pre-aggregate absorbs the volume.

Relatedly, and unintuitively, **there is no co-partitioning optimisation**. Kafka hashes serialized key bytes modulo partition count; Flink hashes the deserialized object's `hashCode()` modulo key-group count. Different function, different modulus. A job keyed by exactly the field Kafka partitioned on still shuffles essentially every record over the network.

6.6 “Batch and streaming are unified” – one API, two behaviours

The same job runs in both modes, and the modes are not equivalent.

	Streaming mode	Batch mode
Shuffles	pipelined	blocking, staged
State	incremental, checkpointed	per-stage, discarded
Failure recovery	from checkpoint	re-run the failed stage
Watermarks	heuristic	advance to infinity at end of input
Results	continuously updated	produced once

A query that is cheap in batch mode can be ruinous in streaming mode – `ORDER BY` over a whole stream, or a regular join whose state is bounded in batch by the input size and unbounded in streaming by time. The unification is real at the level of code and dialect; it is not a promise that the same query is a good idea in both.

6.7 “The runtime handles failures” – recovery is global

Flink recovers automatically, and the default recovery unit is the entire job: one TaskManager lost means every subtask resets to the last checkpoint and replays. Fine-grained recovery exists but applies only where the graph has no pipelined connections between regions, which a shuffle-heavy streaming job generally does not satisfy.

The practical implication is the one noted under Kubernetes: routine cluster operations that are unremarkable for stateless services – node draining, spot reclamation, bin-packing evictions – each cost a full job restart and a replay from the last checkpoint.

6.8 “Backpressure is handled” – it converts into a different failure

Credit-based flow control means a slow sink never loses data; it just slows the source. The leak is that the same mechanism slows barrier propagation, so sustained backpressure lengthens checkpoints until they time out. At that point a job that was merely behind becomes a job that cannot checkpoint, cannot commit transactional output, and – on the next failure – replays from a very old checkpoint. Slow and broken are closer together than they look.

6.9 “The operator manages the job” – it does not understand the job

The Kubernetes operator owns lifecycle. It has no model of what the job computes, and cannot tell you that a regular join's state is growing without bound, that a temporal join is resolving to NULL because log compaction removed the dimension history it needed, or that two sinks share

a transactional-id prefix. Those are properties of the query, visible only in operator metrics and in the output itself – which is the argument for carrying diagnostic columns like `shards_touched` and `subtask_index` all the way through to the serving tables.

Its autoscaler has hard limits worth knowing in advance: it cannot usefully scale a source past the source’s shard count, and it cannot exceed `pipeline.max-parallelism`, which was frozen at first checkpoint.

6.10 “Connectors are interchangeable” – each has its own semantics

`CREATE TABLE ... WITH ('connector' = ...)` presents a uniform surface over systems with entirely different guarantees. Whether a sink is exactly-once, at-least-once or best-effort; whether a source is replayable; whether the connector supports the changelog mode the query produces; whether it is bundled with Flink or versioned independently – all of it varies per connector, and none of it is visible in the query. The uniformity of the DDL is the leak.

7 A short summary

Question	Answer
What problem does it solve	Correct, stateful, event-time computation over data that has not finished arriving
Core mechanism	A parallel dataflow of operators with local keyed state, made durable by asynchronous barrier snapshots
Central abstraction	Event time with watermarks; keyed state scoped by key group; streams and tables as one thing
Why Kubernetes	Flink needs supervised processes, isolation, discovery, HA and elastic workers – and a stateful job needs an operator, because a rolling restart is not a valid upgrade
Hardest genuine limits	Not a serving layer, not an interactive query engine, a latency floor in milliseconds, heavy for small jobs
Leakiest abstractions	Query edits versus savepoint compatibility; watermarks as a heuristic; exactly-once as a sink property; unbounded state by default

The recurring theme across every leak is the same one: **Flink’s abstractions are honest about computation and quiet about time, identity and durability.** What a job computes is well described by its source code. How long its state lives, which operator a saved byte belongs to, when its output becomes visible, and what its clock believes – those are decided elsewhere, and they are where the surprises live.

8 Appendix: where these concepts appear in the reference environment

Concept	Where to look
Job configuration, checkpointing, state backend	charts/flink-stream/templates/flink/_flink-helpers.tpl
Operator UIDs on every stateful operator	datastream/TransactionPipelineJob.java
Watermarks and source idleness	common/Watermarks.java, files/sql/30-windows.sql
Exactly-once Kafka sinks and transactional prefixes	files/sql/30-windows.sql
A sink without transactions, made idempotent instead	sinks/Yugabyte.java (JDBC upsert on the natural key)
Shuffle and skew instrumentation	functions/ShardObserver.java, common/Shards.java
Upgrade mode, savepoints, autoscaler settings	charts/flink-stream/values.yaml
The failure modes above, with symptoms and fixes	docs/09-troubleshooting.md
Aggregation and join mechanics in depth	docs/pdf/aggregation-joins-and-the-operator.pdf